

Explain Grid DP Methods

The key to **2684. Maximum Number of Moves in a Grid** is recognizing that this is a **path DP problem**.

You can move only from column `c` to column `c+1`, so the path always moves left $\rightarrow$ right. That makes the DP direction very clean.

I'll show you the same logic in:

1. Memoization (Top-Down)
2. Tabulation (Bottom-Up)
3. Space Optimization
4. Full dry run on Example 1

1. First understand what DP should store

Suppose we are standing at:

```
text
(row, col)
```

Let:

```
text
dp[row][col] = maximum number of moves possible starting from (row, col)
```

From `(row, col)`, there are at most 3 choices:

```
text
(row-1, col+1)
(row, col+1)
(row+1, col+1)
```

But we can move there only if:

```
text
grid[nextRow][col+1] > grid[row][col]
```

Therefore:

```
text
dp[row][col] =
    1 + max(
        dp[row-1][col+1],
        dp[row][col+1],
        dp[row+1][col+1]
    )
```

for valid increasing moves.

If no move is possible:

```
text
dp[row][col] = 0
```

And because we can start from any row in column 0:

```
text
answer = max(dp[0][0], dp[1][0], ..., dp[m-1][0])
```

2. Memoization — Top Down

We start from every cell in the first column and recursively calculate the answer.

Code

```
C++
class Solution {
public:
    int m, n;
    vector<vector<int>>> dp;

    int solve(vector<vector<int>>& grid, int r, int c) {
        if (dp[r][c] != -1)
            return dp[r][c];

        int ans = 0;
```

```

// Move upper-right
if (r - 1 >= 0 && grid[r - 1][c + 1] > grid[r][c])
    ans = max(ans, 1 + solve(grid, r - 1, c + 1));

// Move right
if (grid[r][c + 1] > grid[r][c])
    ans = max(ans, 1 + solve(grid, r, c + 1));

// Move Lower-right
if (r + 1 < m && grid[r + 1][c + 1] > grid[r][c])
    ans = max(ans, 1 + solve(grid, r + 1, c + 1));

return dp[r][c] = ans;
}

int maxMoves(vector<vector<int>>& grid) {
    m = grid.size();
    n = grid[0].size();

    dp.assign(m, vector<int>(n, -1));

    int ans = 0;

    for (int r = 0; r < m; r++) {
        ans = max(ans, solve(grid, r, 0));
    }

    return ans;
}
};

```

There is one important boundary issue here: `c + 1` must exist.

So a cleaner version is:

```

C++
class Solution {
public:
    int m, n;
    vector<vector<int>> dp;

    int solve(vector<vector<int>>& grid, int r, int c) {
        if (c == n - 1)
            return 0;

        if (dp[r][c] != -1)
            return dp[r][c];

        int ans = 0;

        if (r - 1 >= 0 && grid[r - 1][c + 1] > grid[r][c])
            ans = max(ans, 1 + solve(grid, r - 1, c + 1));

        if (grid[r][c + 1] > grid[r][c])
            ans = max(ans, 1 + solve(grid, r, c + 1));

        if (r + 1 < m && grid[r + 1][c + 1] > grid[r][c])
            ans = max(ans, 1 + solve(grid, r + 1, c + 1));

        return dp[r][c] = ans;
    }

    int maxMoves(vector<vector<int>>& grid) {
        m = grid.size();
        n = grid[0].size();

        dp.assign(m, vector<int>(n, -1));

        int ans = 0;

        for (int r = 0; r < m; r++)
            ans = max(ans, solve(grid, r, 0));

        return ans;
    }
};

```

```
}  
};
```

3. Why does `1 + solve()` work?

This is the most important part.

Suppose:

```
text  
2 → 4 → 9 → 11
```

There are 3 moves.

At 2:

```
text  
solve(2)
```

can move to 4.

So:

```
text  
solve(2) = 1 + solve(4)
```

Then:

```
text  
solve(4) = 1 + solve(9)  
solve(9) = 1 + solve(11)  
solve(11) = 0
```

Therefore:

```
text  
solve(9) = 1  
solve(4) = 2  
solve(2) = 3
```

Notice we're counting **edges/moves**, not cells.

4. Dry Run — Example 1

Given:

```
text  
2  4  3  5  
5  4  9  3  
3  4  2  11  
10 9  13 15
```

Coordinates:

```
text  
(0,0)=2  (0,1)=4  (0,2)=3  (0,3)=5  
(1,0)=5  (1,1)=4  (1,2)=9  (1,3)=3  
(2,0)=3  (2,1)=4  (2,2)=2  (2,3)=11  
(3,0)=10 (3,1)=9  (3,2)=13 (3,3)=15
```

Start at $(0,0)$:

```
text  
2  
↓  
4
```

From $(0,1)=4$:

Possible next cells:

```
text  
(0,2)=3  ✗ 3 < 4  
(1,2)=9  ✓ 9 > 4
```

So:

text

4 → 9

From (1,2)=9:

text

(0,3)=5 ✗
(1,3)=3 ✗
(2,3)=11 ✓

So:

text

9 → 11

Then:

text

11

is in the last column, so no more moves.

Therefore:

text

2 → 4 → 9 → 11

Number of moves:

text

1. 2 → 4
2. 4 → 9
3. 9 → 11

answer = 3

5. Tabulation – Bottom Up

Now let's remove recursion.

Our definition stays exactly the same:

text

dp[r][c] = maximum moves starting from (r,c)

But now we calculate from right → left.

Why?

Because:

text

dp[r][c]

depends on:

text

dp[?][c+1]

So column c+1 must already be calculated.

Therefore:

text

last column
↓
previous column
↓
previous column
↓
...
↓
first column

Code

C++

```
class Solution {  
public:
```

```

int maxMoves(vector<vector<int>>& grid) {
    int m = grid.size();
    int n = grid[0].size();

    vector<vector<int>> dp(m, vector<int>(n, 0));

    for (int c = n - 2; c >= 0; c--) {
        for (int r = 0; r < m; r++) {

            // Upper-right
            if (r - 1 >= 0 &&
                grid[r - 1][c + 1] > grid[r][c]) {
                dp[r][c] = max(dp[r][c],
                               1 + dp[r - 1][c + 1]);
            }

            // Right
            if (grid[r][c + 1] > grid[r][c]) {
                dp[r][c] = max(dp[r][c],
                               1 + dp[r][c + 1]);
            }

            // Lower-right
            if (r + 1 < m &&
                grid[r + 1][c + 1] > grid[r][c]) {
                dp[r][c] = max(dp[r][c],
                               1 + dp[r + 1][c + 1]);
            }
        }
    }

    int ans = 0;

    for (int r = 0; r < m; r++)
        ans = max(ans, dp[r][0]);

    return ans;
}
};

```

6. What does the DP table look like?

For the example:

text

grid:

2	4	3	5
5	4	9	3
3	4	2	11
10	9	13	15

After calculating:

text

dp:

3	2	1	0
0	1	0	0
2	1	1	0
1	1	1	0

For example:

text

dp[0][0] = 3

means:

Starting from 2, the maximum number of moves is 3.

And:

text

```
dp[2][0] = 2
```

means:

text

```
3 → 4 → 9
```

or another valid path of length 2.

Then:

text

```
max(first column)
= max(3, 0, 2, 1)
= 3
```

7. Space Optimization

Now look carefully at our recurrence:

text

```
dp[r][c]
```

only depends on the **next column**:

text

```
dp[r-1][c+1]
dp[r][c+1]
dp[r+1][c+1]
```

We don't need the entire $m \times n$ DP table.

We only need:

text

```
current column
next column
```

So:

text

```
2D DP:  $O(m \times n)$ 
```

can become:

text

```
2 arrays:  $O(m)$ 
```

Space-optimized code

C++

```
class Solution {
public:
    int maxMoves(vector<vector<int>>& grid) {
        int m = grid.size();
        int n = grid[0].size();

        vector<int> next(m, 0);
        vector<int> curr(m, 0);

        for (int c = n - 2; c >= 0; c--) {

            fill(curr.begin(), curr.end(), 0);

            for (int r = 0; r < m; r++) {

                // Upper-right
                if (r - 1 >= 0 &&
                    grid[r - 1][c + 1] > grid[r][c]) {
                    curr[r] = max(curr[r],
                                   1 + next[r - 1]);
                }

                // Right
```

```

        if (grid[r][c + 1] > grid[r][c]) {
            curr[r] = max(curr[r],
                          1 + next[r]);
        }

        // Lower-right
        if (r + 1 < m &&
            grid[r + 1][c + 1] > grid[r][c]) {
            curr[r] = max(curr[r],
                          1 + next[r + 1]);
        }
    }

    next = curr;
}

int ans = 0;

for (int r = 0; r < m; r++)
    ans = max(ans, next[r]);

return ans;
}
};

```

8. Understand curr and next

This is the part you should really understand.

Suppose we're calculating column c .

We have:

text

next column



next[0]
next[1]
next[2]
next[3]

current column



curr[0]
curr[1]
curr[2]
curr[3]

For:

text

curr[r]

we check:

text

next[r-1]
next[r]
next[r+1]

because those represent:

text

upper-right
right
lower-right

Then:

C++

curr[r] = 1 + max(valid next states)

After finishing this column:

```
C++
next = curr;
```

Now the current column becomes the "next column" for the next iteration.

9. Can we optimize further to $O(m)$ with only ONE array?

Yes, but **be careful**.

Because when calculating the current column, you need the **old values** of:

```
text
next[r-1]
next[r]
next[r+1]
```

If you overwrite them directly, you destroy information that another row still needs.

That's why the clean space optimization is:

```
text
curr + next
```

which is:

```
text
O(m)
```

because $2m = O(m)$.

There is no need to force it into one array. That would make the code harder to understand for essentially no asymptotic benefit.

10. Three approaches compared

Approach	Direction	Time	Space
Memoization	Left $\rightarrow$ Right recursion	$O(mn)$	$O(mn)$ + stack
Tabulation	Right $\rightarrow$ Left	$O(mn)$	$O(mn)$
Space optimized	Right $\rightarrow$ Left	$O(mn)$	$O(m)$

The important thing is:

Memoization

```
text
solve(r,c)
  ↓
look at c+1
  ↓
recursively solve
```

Tabulation

```
text
calculate column n-1
  ↓
calculate n-2
  ↓
calculate n-3
  ↓
...
calculate column 0
```

Space optimization

```
text
Only keep:

previous/next column
+
current column
```

The DP pattern you should remember

This problem is a very good example of a general DP pattern:

If a state depends only on the next column, process the matrix from right to left and keep only the necessary columns.

The recurrence is simply:

```
text
dp[r][c] = 1 + max(
    dp[r-1][c+1],
    dp[r][c+1],
    dp[r+1][c+1]
)
```

subject to:

```
text
grid[nextRow][c+1] > grid[r][c]
```

And because the starting position can be anywhere in column 0:

```
text
answer = max(dp[r][0])
```

That's the entire problem. The rest is just choosing whether you want to implement that recurrence using recursion, a 2D table, or two 1D arrays.

Yes. In fact, there is a **cleaner memoization formulation** that I would recommend for learning DP.

Instead of writing three separate `if` statements inside the recursive function, we can store the three possible directions in arrays and loop over them.

Memoization — cleaner version

```
C++
class Solution {
public:
    int m, n;
    vector<vector<int>> dp;

    int solve(vector<vector<int>>& grid, int r, int c) {
        if (c == n - 1)
            return 0;

        if (dp[r][c] != -1)
            return dp[r][c];

        int ans = 0;

        int dr[] = {-1, 0, 1};

        for (int i = 0; i < 3; i++) {
            int nr = r + dr[i];
            int nc = c + 1;

            if (nr >= 0 && nr < m &&
                grid[nr][nc] > grid[r][c]) {
                ans = max(ans, 1 + solve(grid, nr, nc));
            }
        }

        return dp[r][c] = ans;
    }

    int maxMoves(vector<vector<int>>& grid) {
        m = grid.size();
        n = grid[0].size();

        dp.assign(m, vector<int>(n, -1));

        int ans = 0;

        for (int r = 0; r < m; r++) {
            ans = max(ans, solve(grid, r, 0));
        }

        return ans;
    }
};
```

```
}  
};
```

Why this is cleaner

The three moves are:

text

```
      (r-1,c+1)  
        ^  
(r,c) → (r,c+1)  
        ^  
      (r+1,c+1)
```

Instead of writing:

C++

```
if (r - 1 >= 0) ...  
if (...) ...  
if (r + 1 < m) ...
```

we represent the row changes as:

C++

```
int dr[] = {-1, 0, 1};
```

and:

C++

```
nr = r + dr[i];  
nc = c + 1;
```

So the loop automatically checks:

text

```
i = 0 → r-1, c+1  
i = 1 → r,    c+1  
i = 2 → r+1, c+1
```

An even more "DP-style" version

You can also think of the function as:

C++

```
int solve(grid, r, c)
```

meaning:

What is the maximum number of moves I can make if I start from (r,c) ?

Then:

C++

```
int ans = 0;
```

means currently we assume **no move is possible**.

For every valid next cell:

C++

```
ans = max(ans, 1 + solve(grid, nr, nc));
```

The **1** is the move from current cell to next cell.

Finally:

C++

```
return dp[r][c] = ans;
```

stores the answer so we don't calculate the same state again.

And outside:

C++

```
for (int r = 0; r < m; r++)  
    ans = max(ans, solve(grid, r, 0));
```

because we are allowed to start from **any row in column 0**.

One important point

Don't confuse these two DP definitions:

Version A — maximum moves FROM current cell:

```
text
dp[r][c] = maximum moves starting from (r,c)
```

This naturally gives the **right-to-left dependency**:

```
text
dp[r][c] → dp[?][c+1]
```

Version B — maximum moves TO current cell:

```
text
dp[r][c] = maximum moves needed to reach (r,c)
```

That would naturally be calculated **left-to-right**.

Both are valid. For this problem, I prefer **Version A** because the recurrence is much easier to understand:

```
text
current cell
  ↓
choose best valid next cell
  ↓
1 + answer(next cell)
```

That's the memoization pattern you should remember.

Yes. If we keep **exactly the same DP definition** as the memoization version:

```
dp[r][c] = maximum number of moves possible starting from (r,c)
```

then the tabulation version is almost the same logic — we just remove recursion.

Tabulation version

```
C++
class Solution {
public:
    int maxMoves(vector<vector<int>>& grid) {
        int m = grid.size();
        int n = grid[0].size();

        vector<vector<int>> dp(m, vector<int>(n, 0));

        int dr[] = {-1, 0, 1};

        // Last column = 0 moves
        // So start from second-last column
        for (int c = n - 2; c >= 0; c--) {

            for (int r = 0; r < m; r++) {

                int ans = 0;

                for (int i = 0; i < 3; i++) {

                    int nr = r + dr[i];
                    int nc = c + 1;

                    if (nr >= 0 && nr < m &&
                        grid[nr][nc] > grid[r][c]) {

                        ans = max(ans, 1 + dp[nr][nc]);
                    }
                }

                dp[r][c] = ans;
            }
        }

        int ans = 0;

        // Can start from any cell in first column
```

```

    for (int r = 0; r < m; r++) {
        ans = max(ans, dp[r][0]);
    }

    return ans;
}
};

```

How memoization changed into tabulation

Your memoization was:

```

C++
int solve(grid, r, c)

```

and inside:

```

C++
ans = max(ans, 1 + solve(grid, nr, nc));

```

So the dependency was:

```

text
dp[r][c]
  ↓
dp[nr][c+1]

```

Therefore, in tabulation, we must calculate:

```

text
column n-1
  ↓
column n-2
  ↓
column n-3
  ↓
...
column 0

```

That's why:

```

C++
for (int c = n - 2; c >= 0; c--)

```

We don't actually need to calculate the last column because:

```

text
dp[r][n-1] = 0

```

There is nowhere to move.

The important correspondence

Memoization:

```

C++
ans = max(ans, 1 + solve(grid, nr, nc));

```

becomes tabulation:

```

C++
ans = max(ans, 1 + dp[nr][nc]);

```

That's basically the entire conversion.

So remember:

MEMOIZATION			TABULATION	
<code>solve(r,c)</code>			<code>dp[r][c]</code>	
↓			↓	
<code>solve(nr,c+1)</code>	→		<code>dp[nr][c+1]</code>	
↓			↓	
recursion			already calculated	

The DP definition doesn't change. Only the way we calculate the states changes.

Complexity

text

Time = $O(m \times n \times 3) = O(mn)$

Space = $O(m \times n)$

And this is a very good example to learn the conversion:

Memoization → **identify dependency** → **reverse the dependency** → **write tabulation**.